

ICS-PA3-3 实验报告

241220029 杨智宇

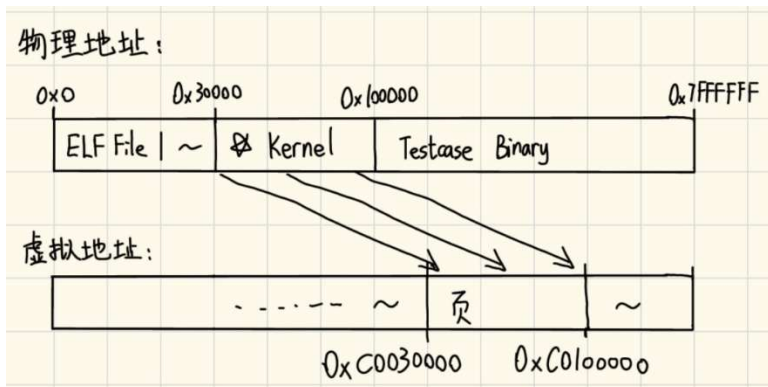
思考题

1. Kernel 的虚拟页和物理页的映射关系是什么？请画图说明；

答：Kernel 会将自己的内存映射到所有进程的虚拟地址空间的 0xc0030000 及以上区域中，这在 start.S 代码中有所体现，代码如下：

```
#ifdef IA32_PAGE
#    define KOFFSET 0xc0000000
#    define va_to_pa(x) (x - KOFFSET)
#else
#    define va_to_pa(x) (x)
#endif
```

映射关系如下图所示：



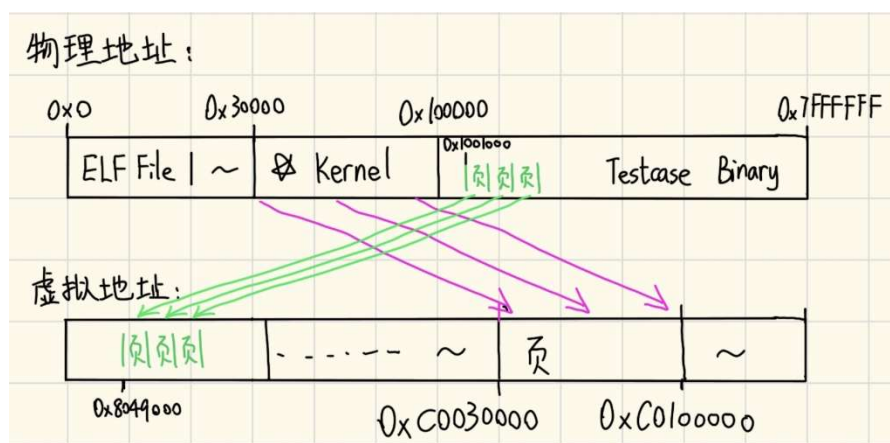
2. 以某一个测试用例为例，画图说明用户进程的虚拟页和物理页间映射关系又是怎样的？Kernel 映射为哪一段？你可以在 loader() 中通过 Log() 输出 mm_malloc 的结果来查看映射关系，并结合 init_mm() 中的代码绘出内核映射关系。

答：在 loader() 中添加 log() 后，可以在 make test 后观察到以下输出：

```
./nemu/nemu --autorun --testcase mov-c --kernel
NEMU load and execute img: ./kernel/kernel.img elf: ./testcase/bin/mov-c
nemu trap output: [src/main.c,82,init_cond] {kernel} Hello, NEMU world!
nemu trap output: [src/elf/elf.c,29,loader] {kernel} ELF loading from ram disk.
nemu trap output: [src/elf/elf.c,44,loader] {kernel} ELF phy_addr: 0x1000000 ,vaddr: 0x0 .
nemu trap output: [src/elf/elf.c,44,loader] {kernel} ELF phy_addr: 0x1001000 ,vaddr: 0x8049000 .
nemu trap output: [src/elf/elf.c,44,loader] {kernel} ELF phy_addr: 0x1002000 ,vaddr: 0x804a000 .
nemu trap output: [src/elf/elf.c,44,loader] {kernel} ELF phy_addr: 0x1003000 ,vaddr: 0x804c000 .
nemu: HIT GOOD TRAP at eip = 0x080490c2
NEMU2 terminated
./nemu/nemu --autorun --testcase mov-jcc --kernel
NEMU load and execute img: ./kernel/kernel.img elf: ./testcase/bin/mov-jcc
nemu trap output: [src/main.c,82,init_cond] {kernel} Hello, NEMU world!
nemu trap output: [src/elf/elf.c,29,loader] {kernel} ELF loading from ram disk.
nemu trap output: [src/elf/elf.c,44,loader] {kernel} ELF phy_addr: 0x1000000 ,vaddr: 0x0 .
nemu trap output: [src/elf/elf.c,44,loader] {kernel} ELF phy_addr: 0x1001000 ,vaddr: 0x8049000 .
nemu trap output: [src/elf/elf.c,44,loader] {kernel} ELF phy_addr: 0x1002000 ,vaddr: 0x804a000 .
nemu trap output: [src/elf/elf.c,44,loader] {kernel} ELF phy_addr: 0x1003000 ,vaddr: 0x804c000 .
nemu: HIT GOOD TRAP at eip = 0x080490c1
NEMU2 terminated
```

(此处仅展示出了前两个用例，后面的测试用例都是如此)

因此容易画出映射关系，如下图：



3. “在 Kernel 完成页表初始化前，程序无法访问全局变量”这一表述是否正确？

在 `init_page()` 里面我们对全局变量进行了怎样的处理？

答：PE = 1 且 PG = 1 时，“在 Kernel 完成页表初始化前，程序无法访问全局变量”这一表述是正确的。因为在页表初始化之前，线性地址还不能被转换成物理地址，直接访问位于某些特定的物理地址上的全局变量肯定是不可能的。

`init_page()` 函数建立了正确的页表条目，使全局变量的虚拟地址可以正确地映射到它们对应的物理地址。